Meshaal Mouawad
Mississippi State University, Electrical and Computer Engineering Department,
This is a summary report of my research assignments I was given by my advisor during my PhD program

SUMMARY REPORT: FOR RESEARCH ASSIGNMENTS OF MY PHD

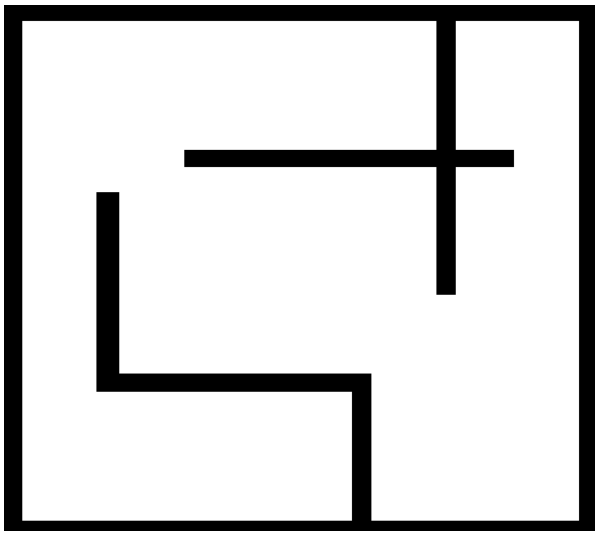
WEEK 11

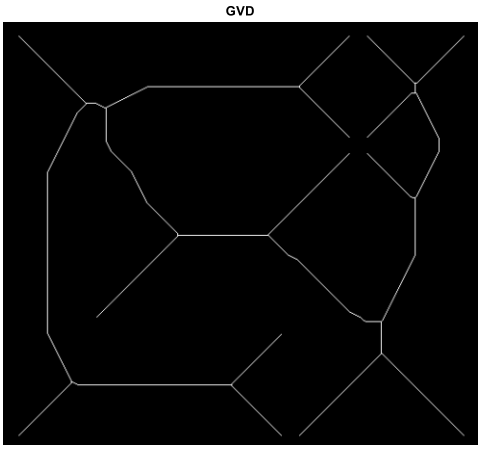
Summary:

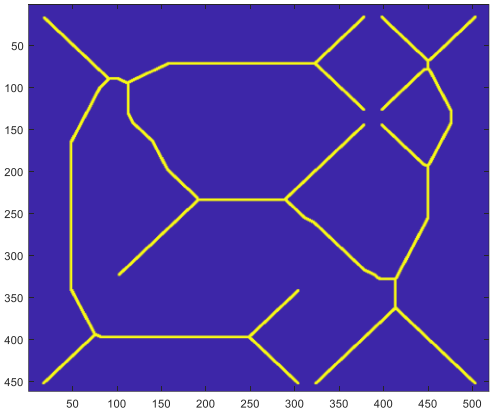
I. TSP Algorithms and DWA Method

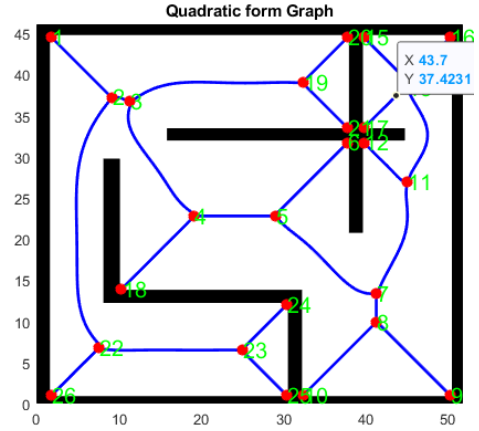
Assignment: 1. TSP (Travelling salesman problem)

Completed: Yes
Submitted:
Problems: TSP matlab code is not working
Explain: NA
Procedure: I used figure'Map4_3.png, from the 1st a assignment





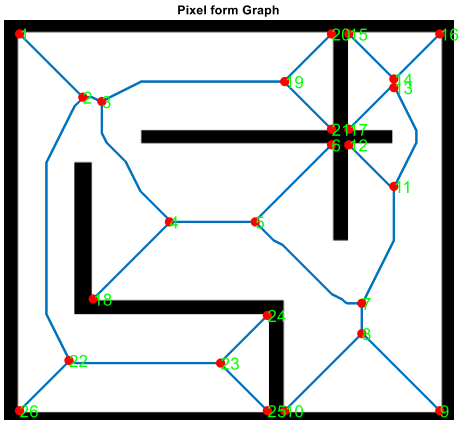


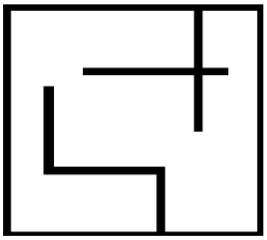


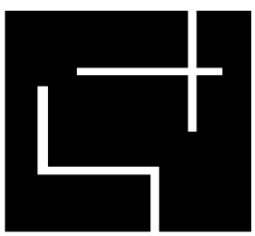
1 I run MATLAB Code to get the Adjacent Matrix :

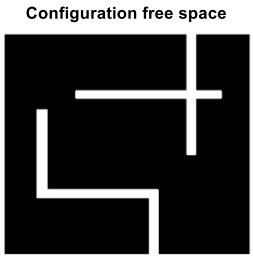
This Report was submitted in 2019 for Mississippi State University, Electrical and Computer Engineering Department during Fall 2019 foe the course of Introduction to Radars Engineering instructed by Dr. Ball John

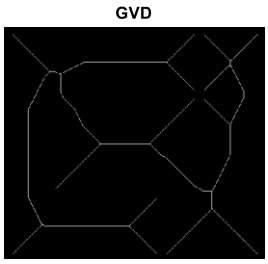
Meshaal Mouawad, Mississippi State University, Starkville, MS 39759 USA (e-mail: mm4922@msstate.edu).












```
add node(2)
add node(3)
add node(4)
add node(5)
add node(6)
add node(7)
add node(8)
add node(9)
add node(10)
add node(11)
add node(12)
add node(13)
add node(14)
add node(15)
add node(16)
add node(17)
add node(18)
add node(19)
add node(20)
add node(21)
add node(22)
add node(23)
add node(24)
add node(25)
add node(26)
finish
```

2 I took the Adjacency Matrix which is located in the QUADGRAPH > *edge_matrix* in the output variables using this command:

- y= QUADGRAPH.edge_matrix

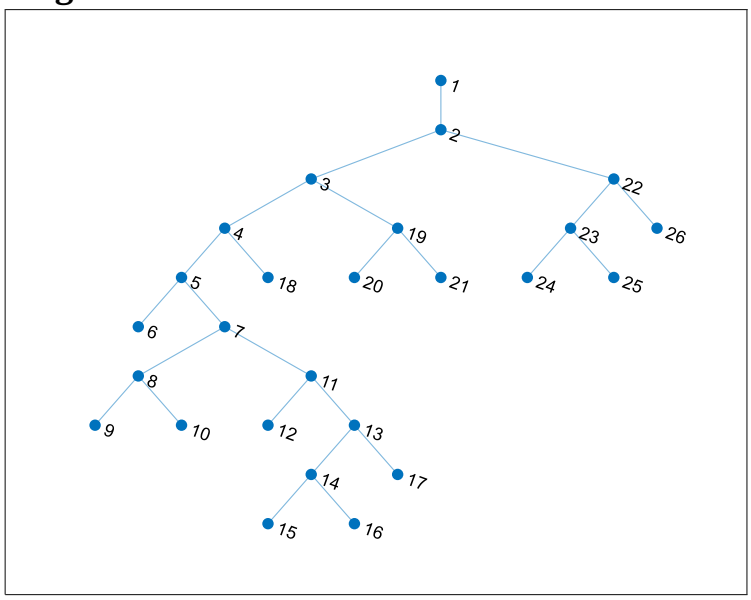
3 I converted the edge_matrix to Sparse using the command:

- B = Sparse(y)
  Then I saved B as .mat file

I have not completed it as Tinjgun told me the adjacency matrix is not correct

and convert it to Sparse then use it in Python.

To Visualize the map with the path edges this is how the distant matrix looks like Using MATLAB



```
GraphSparse =

  graph with properties:

    Edges: [25×2 table]
    Nodes: [26×0 table]

plot_A =

  GraphPlot with properties:

     NodeColor: [0 0.4470 0.7410]
    MarkerSize: 4
        Marker: 'o'
     EdgeColor: [0 0.4470 0.7410]
     LineWidth: 0.5000
```

```
       LineStyle: '-'
       NodeLabel: {1×26 cell}
       EdgeLabel: {}
           XData: [1×26 double]
           YData: [10 9 8 7 6 5 5 4 3 3 4 3
3 2 1 1 2 6 7 6 6 8 7 6 6 7]
           ZData: [0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

Tingiun Instructions:
Sunday 11:32 PM] Lei, Tingjun
Then, you only need to do is change the cost matrix and nodeNum
[Sunday 11:32 PM] Lei, Tingjun

That's almost the same since Adj matrix scale the connected path to 1 and cost/distance matrix has the "distance" value instead of 1

BTW, as far as I know, you now first use the Voronoi diagram to determine the feasible path in the map. When there are target points that need to be reached, each target point should have a corresponding distance in the TSP that can be connected instead of unable to connect. (NodeNum in python code should be starting point + targets number)

The mat you have now is based on the Voronoi diagram. It's not the right cost matrix that we need to run the TSP.

You have to figure out by yourself. You need to do is first calculate the path length (not the euclidean distance but the real path length according to the VD) between each "node". Please note, these nodes are in your Adj matrix. Then when you have targets information. You send the targets location and get the cost matrix of the TSP (distance according to path length) and then you run your python code.

Appendix